

The 3 AM Pager Is Solved

Building an Autonomous SRE Agent with
Google's ADK

[TheNightOps](#) · built with Google ADK & Gemini

Mehul Patel · DevOps Team Lead, BuildingMinds

GDE (Cloud) · Docker Captain · Grafana & Friends Berlin

@nomadicmehul · GC summit Czech Republic 2026 · Prague, June 09





Mehul Patel

DevOps Team Lead at
BuildingMinds

Google Developer Expert in Google Cloud

Docker Captain

Grafana & Friends Meetup Organizer, Berlin

SRE · Kubernetes · AI-powered operations ·
Open source

Agenda

- | | | |
|----|---------------------|---|
| 01 | The Problem | Why traditional alerting falls short |
| 02 | What is ADK? | Google Agent Development Kit overview |
| 03 | Architecture | Multi-agent design & MCP integration |
| 04 | Deep Dive | Agents, MCP servers, orchestration patterns |
| 05 | Live Demo | Triggering failures & watching agents investigate |
| 06 | Takeaways | What we learned building with ADK |

3 A.M. CONFESSIONS

Me at 3 AM:

“I should automate this.”

Also me at 3 AM:

manually SSHing into 14 pods

SECTION 01

The Problem

Why traditional incident response doesn't scale



The 3 AM Problem

70%

Alert fatigue rate in SRE teams

45 min

Average time to first meaningful action

60%

Incidents repeat with the same root cause

Alert → Triage → Investigate → Escalate → Fix → Repeat

What if an AI agent could handle the investigation autonomously?

EXPECTATION VS REALITY

SRE Job Description:

“Automate everything.”

SRE Reality:

kubectl get pods -A | grep -v Running

...at 3 AM, for the 47th time this quarter.

What is Google ADK?

Agent Development Kit — building blocks for
multi-agent systems



ADK Python v2.0.0 GA release

ADK Python 2.0 is released for general availability as of May 19, 2026.



Google ADK at a Glance

Open-source framework for building multi-agent AI systems

Open Source

Apache 2.0

SDKs

Python, Go, TS, Kotlin & Java

Model-Agnostic

Gemini, Claude, GPT-4o

Production-Ready

Built-in eval & debugging

Multi-Agent Orchestration

Parallel, sequential & hierarchical orchestration with structured handoffs

Native MCP Support

Connect official & custom MCP servers for tool integration

Streaming & Real-time

Bidirectional audio/video streaming with real-time interaction

Local Dev & Deploy

CLI + Web UI for development; deploy to Cloud Run or GKE

Deep GCP Integration

ADK fits naturally into the Google Cloud ecosystem

Compute

**Cloud Run, GKE,
Compute Engine**

Serverless & container
deployment

Data

**BigQuery, AlloyDB,
Cloud Spanner, Postgres**

Storage & analytics integration

Operations

**Cloud Monitoring,
Cloud Logging, Security**

Observability & compliance
built-in

+ Apigee for API management · Vertex AI for model serving · Cloud IAM for security

ADK Development Workflow

From local development to production in three steps



Why ADK Matters

Unified enterprise AI framework

All GCP services, no duplication

Enterprise app connectivity

Production-ready from day one

Why ADK for SRE?

Traditional Alerting

- Static threshold rules
- Manual investigation per alert
- Context switching between tools
- No memory of past incidents
- Linear, single-threaded process

ADK-Powered Agent

- AI-driven pattern recognition
- Parallel multi-agent investigation
- Unified tool access via MCP servers
- TF-IDF incident memory & matching
- Orchestrated, concurrent workflows

Incident Response

Expectation

1. Get alert
2. Check dashboard
3. Find root cause
4. Fix it
5. Go back to sleep

Reality

1. Get alert
2. Which cluster was this again?
3. VPN dropped
4. Wrong terminal — that was prod
5. Slack: “is anyone else seeing this?”
6. It was DNS. It’s always DNS.
7. Sun is rising



PRESENTER REMINDER

Start the GKE Cluster

Spin up the demo cluster BEFORE you begin

```
$ gcloud container clusters get-credentials nightops-demo
```



Architecture

Multi-agent design with ADK orchestration &
MCP tools

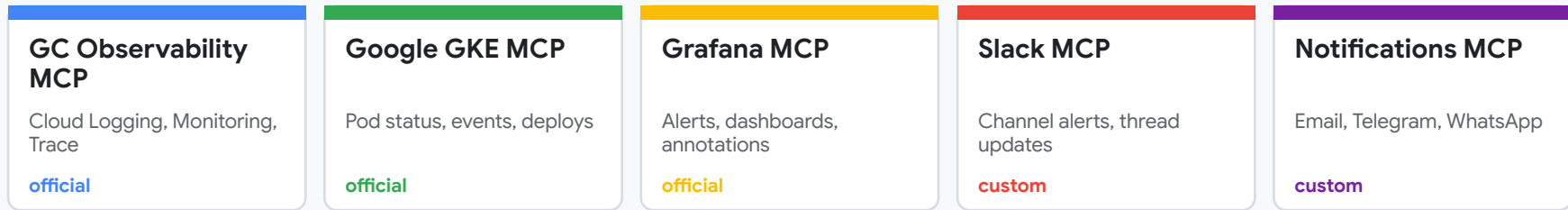


Plan A — The Dream

Full multi-agent MCP architecture: 1 orchestrator + 5 sub-agents + 5 MCP servers

ADK Root Orchestrator + 5 Sub-Agents · Powered by Gemini

↓ all tools via MCP ↓



Reality check: GKE & Observability MCPs are in preview/beta. In testing...

MCP connections hanging/timing out · ADK TaskGroup errors on MCP failures · investigations stuck at "Triage" · httpx connection-pool issues

Plan B — Simple Mode

Working today: 1 agent + 10 kubectl FunctionTools — no MCP, no sub-agents

INGESTION · Grafana / Alertmanager / PagerDuty → Webhook Receiver → Dedup



ADK AGENT — Single Agent (Gemini Pro)

no orchestrator, no sub-agents

▼ kubectl subprocess calls ▼

get_pods

get_events

get_deployments

top_pods

get_pod_logs

describe_pod

get_deploy_history

get_namespaces

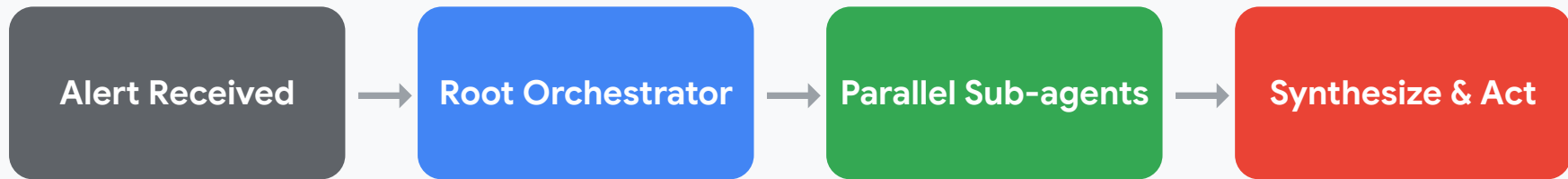
get_nodes

get_resource_yaml

Read-only RCA — no write operations

Root Orchestrator

The brain of the operation — powered by Gemini



How it works

Receives the deduplicated alert with fingerprint and context

Delegates to Log Analyst, Deployment Correlator & Runbook Retriever in parallel

Synthesizes findings using Gemini's reasoning, then drafts the RCA and notifies

Specialized Sub-Agents

1

Log Analyst

Queries GC Observability MCP for error patterns, stack traces and anomalies

2

Deployment Correlator

Uses GKE MCP to check pod health, events, deployments and what changed

3

Runbook Retriever

Searches Grafana MCP for alerts, dashboards and past incident runbooks

4

Communication Drafter

Generates RCAs and sends via Slack MCP + Notifications MCP

MCP Servers: The Tool Layer

Model Context Protocol — how agents talk to infrastructure

Official MCP servers

Google Cloud Observability

Logging, Monitoring, Trace — remote, IAM auth

Google GKE

Pod status, events, deployments — remote

Grafana

Alerts, dashboards, annotations — stdio, token

Custom MCP servers

Slack MCP

Post incident alerts, thread updates, channel notifications — custom built

Notifications MCP

Email (SMTP), Telegram & WhatsApp multi-channel delivery — custom built

“How many agents does your system have?”

Yes.

Root Orchestrator

Log Analyst

Deployment Correlator

Runbook Retriever

Communication Drafter

Anomaly Detector

Intelligence Layer

Incident Memory

TF-IDF vectorization of past incidents

Cosine similarity matching

Learns from every investigation

Surfaces relevant runbooks automatically

Policy Engine — 4 approval levels

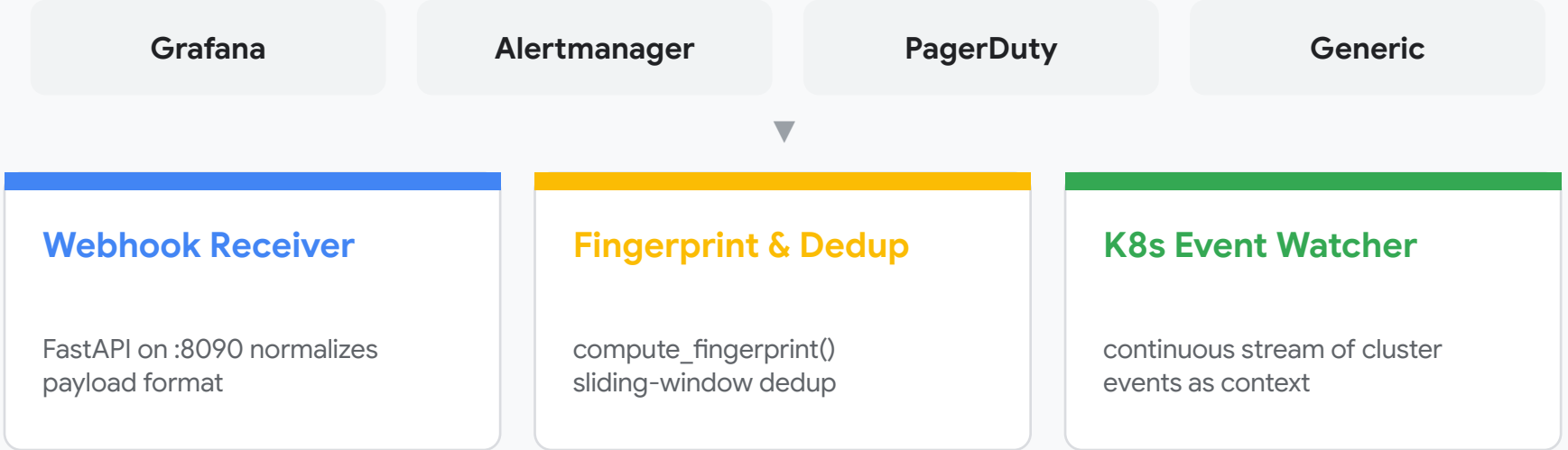
Auto-approve — safe actions (restart pod)

Env-gated — auto in dev, manual in prod

Always-approve — human required

Blocked — never auto-execute

Ingestion Pipeline



THE MCP DIFFERENCE

When official MCP servers exist for GKE & Observability:

“npm install and done!”

When you need Slack + Notifications MCP:

“It ain’t much, but it’s honest work.”

(400 lines of Python later...)

BEFORE & AFTER MCP

Before MCP:

“Let me write 47 custom API integrations.”

After MCP:

```
npm install @google-cloud/gcloud-mcp
```

Official Google Cloud + Grafana MCPs = plug and play

A MOMENT OF SILENCE...

...for the Demo Gods.

“Please work. Please work. Please work.”

It worked on my machine. Yesterday. Once.

SECTION 04

Live Demo

Let's break things and watch the agents investigate



Demo Scenarios

- | | | |
|---|--------------------------|--|
| 1 | memory-leak | Gradual memory increase until OOM |
| 2 | cpu-spike | Sudden CPU utilization spike |
| 3 | cascading-failure | Multi-service dependency chain failure |
| 4 | config-drift | Configuration mismatch detection |
| 5 | oom-kill | Container killed by kernel OOM |

Demo Commands

Quick reference for the live demo

Start agent

```
$ nightops agent watch
```

Interactive

```
$ nightops agent run --interactive
```

Trigger

```
$ ./scripts/demo-gke.sh --scenario cpu-spike
```

Dashboard

```
$ nightops dashboard
```

Verify

```
$ nightops verify
```

What to Watch For

Key moments during the live demo

1 Alert arrives via webhook

Watch the dedup engine filter noise

2 Orchestrator delegates

Sub-agents spin up in parallel

3 MCP servers query infra

GKE, Observability & Grafana fetch live data

4 RCA generation

Gemini synthesizes findings into root cause

5 Notification sent

Slack MCP + Notifications MCP deliver the report

THE DIFFERENCE

Production:

“This is fine.”

TheNightOps:

“Already identified root cause, drafted the RCA, and notified the team. Go back to sleep.”

The Bottom Line

Manual investigation

45 min

mean time to resolution

With TheNightOps

< 3 min

mean time to resolution

...and your SRE actually got to sleep.

Key Takeaways

1

ADK makes multi-agent systems accessible

Clean abstractions for orchestration, sub-agents and tool integration

2

MCP is the universal tool protocol

Official Google Cloud & Grafana MCPs + custom servers = plug and play

3

Gemini excels at reasoning over SRE data

Pattern recognition across logs, events and deployment history

4

Start with guardrails, not YOLO

Policy Engine ensures safe, graduated automation from day one

The real takeaway from this talk:

**Let the agents handle 3 AM.
You deserve sleep.**

(Your on-call rotation will thank you.)

Get Started

1 Clone & Install

```
git clone github.com/nomadicmehul/TheNightOps
```

2 Configure

```
cp config/.env.example config/.env
```

3 Verify

```
nightops verify
```

4 Run Demo

```
./scripts/demo-gke.sh --scenario memory-leak
```

5 Explore

```
nightops agent run --interactive
```

Thank You

Questions & Discussion

Mehul Patel — DevOps Team Lead at BuildingMinds

GDE (Google Cloud) · Docker Captain · Grafana & Friends

github.com/nomadicmehul/TheNightOps · nomadicmehul.dev · [@nomadicmehul](https://twitter.com/nomadicmehul)

